

به نام خدا

پروژه بازی سازی درس برنامه نویسی پیشرفته

بهار 1405

مقدمه

سلام به داک پروژه خوش اومدید. سندی که قراره حسابی بهش نگاه کنید، سر بخش های مختلفش تو سر و کله هم دیگه بزنین و به خودتون که این پروژه رو انتخاب کردید و کسایی که این داک رو نوشتند حسابی فحش بدید.

امیدواریم که این پروژه جدای از پروسه یادگیری (که خیلی مهمه) براتون لذت بخش باشه و بعدا وقتی بهش نگاه می کنین حسابی از ساختن یه بازی کامل با دوستتون کیف کنید:

سرفصل ها:

کانسپت کلی

ساختار کلی

زمان تحویل و فاز های پروژه:

- فاز اول
- فاز دوم
- فاز سوم (غول مرحله آخر)

توضیح کامل مکانیک ها

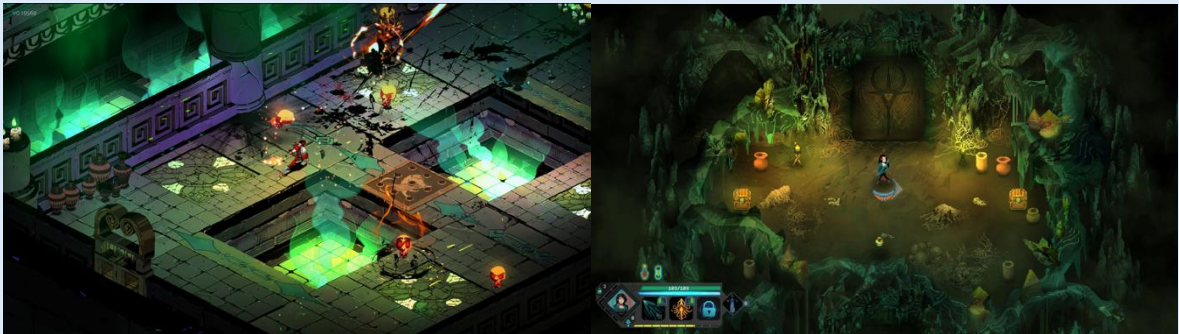
موارد امتیازی

توصیه ها

کانسپت کلی:

کانسپت (Concept) و هدف کلی پروژه ساخت یک بازی 2d top-down در سبک action-adventure به صورت دو نفره (co-op) هست.

اصطلاح **2D Top-Down** به نحوه قرار گیری دوربین بازی گفته می شود که در آن دوربین از بالا، تمام یا بخشی از نقشه بازی را نشان می دهد این دوربین در بسیاری بازی ها در سبک های مختلفی به کار می رود؛ از معروف ترین آنها می توان به بازی های Children of Morta و Hades اشاره کرد.



سبک بازی **Action-Adventure Co-op**:

بخش **Action** به این معنا که بازی شامل مکانیک های کنش محور مانند مبارزه، حرکت، واکنش سریع می باشد.

بخش **Adventure** به وجود اکتشاف، حل پازل، پیشروی در محیط، تعامل با جهان بازی و گاهی عناصر روایی اشاره دارد-Co-op . یعنی بازی به گونه ای طراحی شده که دو بازیکن بتوانند به صورت همکاری محور مراحل، چالش ها یا پازل ها را با همکاری یکدیگر پیش ببرند.

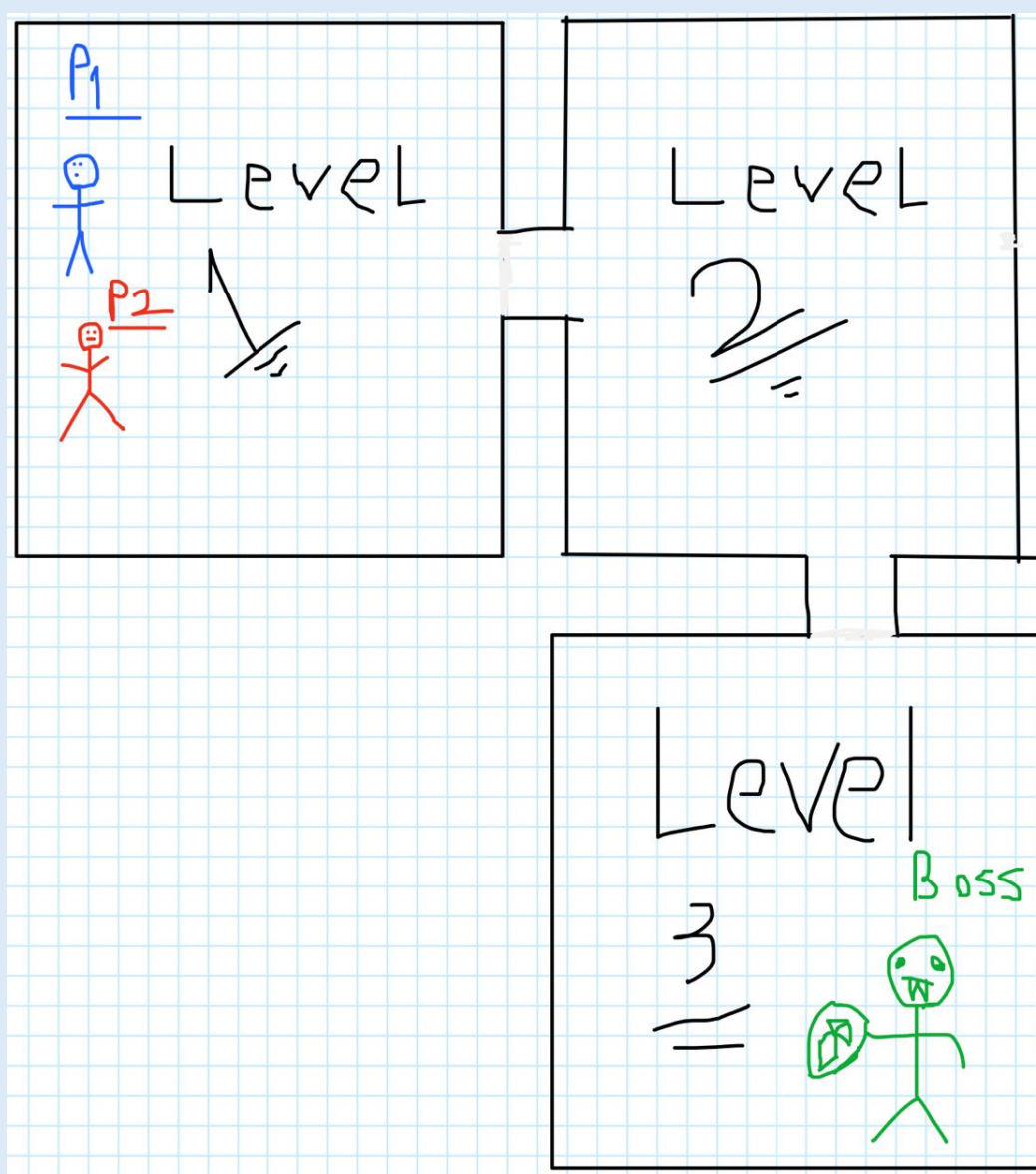


ساختار کلی:

پروژه یک بازی دونفره در سه مرحله (Level) است. مرحله اول، جایی است که بازیکنان از آن بازی را شروع می‌کنند. این مرحله از پازل‌های مختلف تشکیل شده است.

مرحله دوم شامل دشمن‌های (Enemies) مختلف است. این مراحل از طریق درهایی به یکدیگر متصل شده‌اند و بازیکنان تنها پس از پشت سر گذاشتن چالش‌های هر مرحله، کلید مربوط به در را پیدا می‌کنند و می‌توانند وارد مرحله بعد شوند.

در مرحله نهایی، بازیکنان با یک باس‌فایت (Boss Fight) روبه‌رو می‌شوند و با شکست دادن باس، بازی به پایان می‌رسد.



تحويل و فاز های پروژه

پیاده‌سازی بازی در قالب سه فاز زمان‌بندی شده انجام می‌شود.

فاز اول:

طراحی مراحل بازی و دریافت فیدبک از TA ها

ددلاین: ۲۳ خرداد

در این فاز، ابتدا باید ایده و کانسپت (Concept) کلی بازی خود را مشخص کنید.

سپس طراحی بازی (Game Design) و طراحی مرحله اول (Level Design) بازی و پازل‌های آن را انجام دهید. خروجی را به صورت کامل روی کاغذ یا سند طراحی کنید؛ طوریکه اگر این طراحی به شخصی ناشناس نشان داده شود، بتواند بر اساس آن بازی را پیاده‌سازی کند. پس از ارائه سند طراحی بازی (Game Design Document)، تیم‌ها آن را بررسی کرده و پیشنهادهای و انتقادات خود را به شما ارائه می‌دهند.

فاز دوم:

تولید Level 1,2

ددلاین: ۳ تیر

در فاز دوم، شما باید مراحل ۱ و ۲ بازی را تولید کنید. برای پیاده‌سازی این مراحل، نیاز به طراحی و توسعه‌ی بخش‌های مختلف بازی خواهید داشت:

1) ساخت بازیکن‌ها و مکانیک‌های مربوط به آن‌ها:

- طراحی دو کاراکتر قابل بازی
- طراحی Input System
- طراحی Movement System
- طراحی Combat System
- پیاده‌سازی Health System
- پیاده‌سازی منطق Death و Game Over
- طراحی و پیاده‌سازی Animations و Audio

2) ساخت دشمنان:

دشمن‌ها می‌توانند منطق‌های متفاوتی داشته باشند، اما به‌طور کلی به دو دسته تقسیم می‌شوند:

- **Following Enemy**
- **Shooting Enemy**

هم‌چنین هرکدام از دشمنان باید ویژگی‌های مشترک زیر را داشته باشند:

- **Health and Death System**
- **Audio and Animations**

3 (Collectible ها):

Collectible ها آیتم‌هایی هستند که معمولاً از دشمنان (**Enemies**) دراپ می‌شوند و تغییراتی در روند بازی ایجاد می‌کنند.

برای مثال، **Health Potion** در بسیاری از بازی‌ها باعث افزایش HP بازیکن می‌شود.

انواع Collectible های بازی:

- نمونه‌ای از Collectible های بازی:
- **Increase Health**
- **Temporary Buffs**

آیتم‌ها می‌توانند براساس خلاقیت شما طراحی و پیاده‌سازی شوند.

4 (Puzzle ها):

پازل‌هایی که بازیکن در طول بازی با آن‌ها روبرو می‌شود، می‌توانند براساس خلاقیت شما طراحی شوند برای مثال:

- ☐ **Spikes / Traps**
- ☐ **Trigger Puzzles**

5 (Map):

با استفاده از **Tile Map**، محیط (**Environment**) و نقشه‌ی (**Map**) بازی را که در فاز اول طراحی کرده بودید، پیاده‌سازی و ایجاد کنید.

فاز سوم:

تولید Level 3 و ارائه‌ی خروجی نهایی قابل قبول

ددلاین: ۲۰ تیر

پس از عبور از Level 2، بازیکنان وارد Level 3 می‌شوند؛ جایی که با غول مرحله‌ی آخر (Boss Fight) روبه‌رو خواهند شد. در فاز سوم و نهایی پروژه، وظیفه‌ی شما طراحی و پیاده‌سازی مرحله‌ی آخر و باس‌فایت است.

همچنین در این فاز باید بخش‌های تکمیلی بازی، مانند UI/UX، صفحه‌ی منو (Menu)، سیستم ذخیره‌سازی بازی (Save System) و خروجی نهایی قابل بازی (Build) را نیز پیاده‌سازی کنید.

موارد مورد نیاز برای پیاده‌سازی:

- Boss Fight
- Menu & UI/UX
- Save System
- Final Playable Build



تعریف و توضیح مکانیک ها:

1) کاراکتر های قابل بازی:

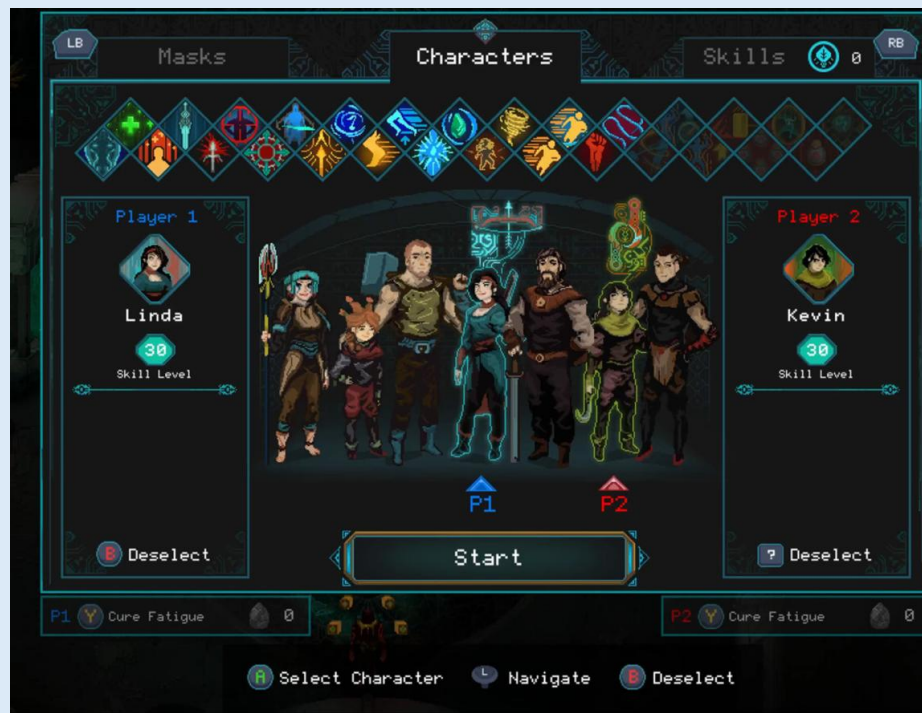
دو کاراکتر بازی شامل **Melee Player** و **Ranged Player** خواهند بود.

Melee Player

این کاراکتر با استفاده از سلاح‌هایی مانند شمشیر، مشت یا سلاح‌های مشابه (براساس خلاقیت شما) می‌تواند به دشمنان حمله کند. همچنین در برابر حملات دشمن مقاومت بیشتری دارد و **Damage** کمتری دریافت می‌کند.

Ranged Player

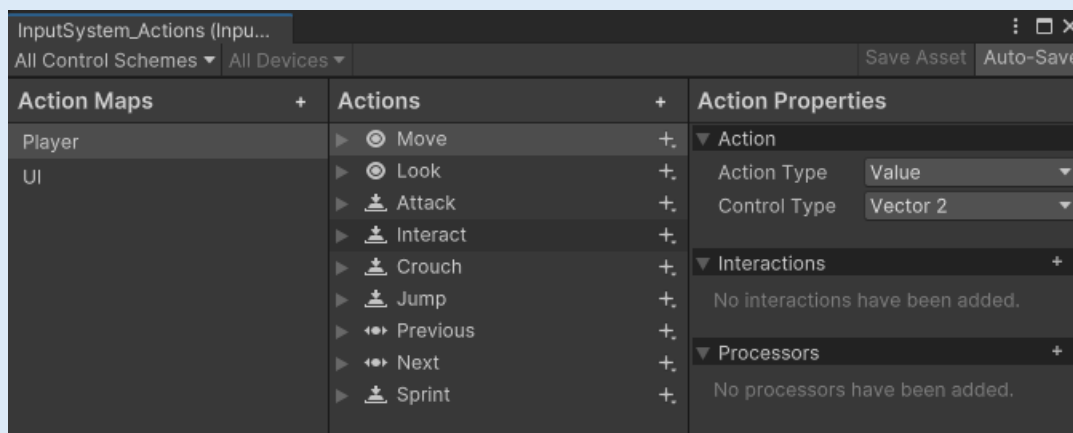
این کاراکتر با استفاده از سلاح‌هایی مانند کمان، تفنگ یا سلاح‌های مشابه (براساس خلاقیت شما) می‌تواند از فاصله دور به دشمنان حمله کند. در مقابل، مقاومت کمتری نسبت به حملات دشمن دارد و **Damage** بیشتری دریافت می‌کند.



Input system (2)

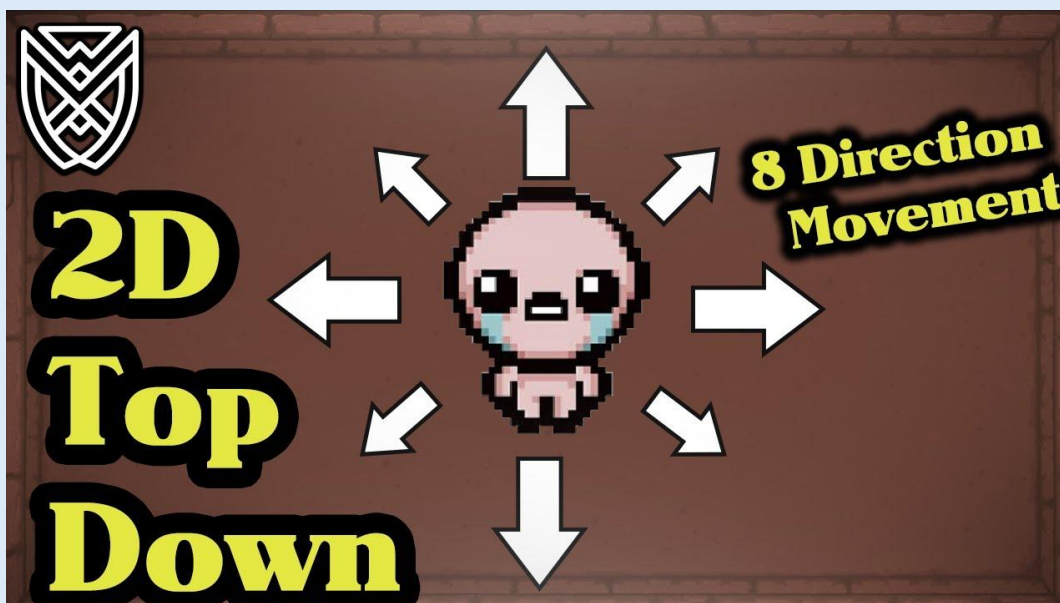
بازی باید از کنترل همزمان دو بازیکن روی یک سیستم پشتیبانی کند. کلیدهای کیبورد برای هر بازیکن باید مجزا باشند؛ برای مثال، می‌توان از کلیدهای **WASD** برای حرکت بازیکن اول و از **Arrow Keys** برای حرکت بازیکن دوم استفاده کرد. انتخاب و تنظیم دقیق دکمه‌های هر بازیکن بر عهده‌ی شماست.

دقت کنید حتماً از **New Input system** یونیتی برای مدیریت کنترلر هر دو بازیکن استفاده کنید.



Movement system (3)

هر دو بازیکن باید بتوانند براساس ورودی‌های کیبورد، در چهار جهت اصلی یعنی چپ، راست، بالا و پایین و همچنین ترکیب این جهت‌ها حرکت کنند. حرکت بازیکنان باید نرم، روان و قابل کنترل باشد.



combat system (4)

Melee Player باید بتواند با زدن یک دکمه، سلاح خود را حرکت دهد و به دشمنان (**Enemies**) آسیب (**Damage**) وارد کند.

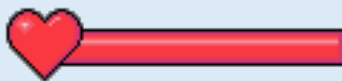


Ranged Player نیز باید بتواند با زدن یک دکمه، از سلاح خود تیر شلیک کند. تیر پس از برخورد با دشمنان (**Enemies**) باید به آن‌ها آسیب (**Damage**) وارد کند.



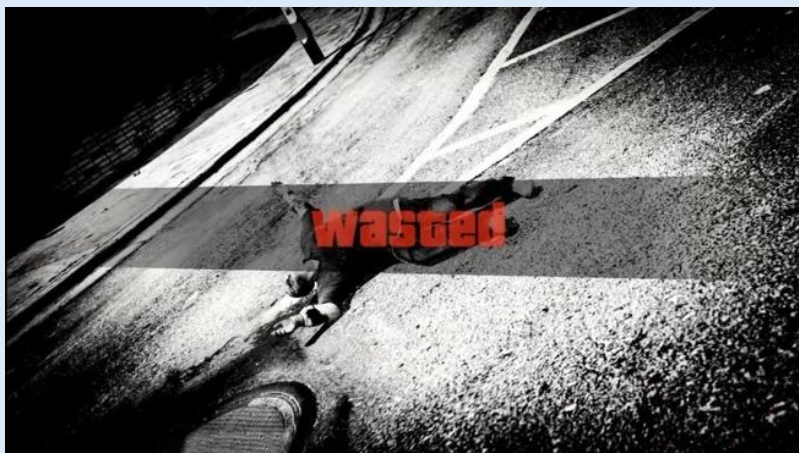
Health system (5)

هر بازیکن باید یک نوار سلامت (**Health Bar**) اختصاصی و سه جان (**Health Points**) داشته باشد. با خالی شدن کامل نوار سلامت، یکی از جان‌های بازیکن کم می‌شود و نوار سلامت دوباره پر می‌شود.



Death and Game Over Logic (6)

اگر تعداد جان‌های (**Health Points**) یکی از بازیکنان به پایان برسد، آن بازیکن می‌میرد و بازی بلافاصله **GameOver** می‌شود. در این حالت، باید امکان اجرای دوباره‌ی بازی (**Restart**) برای بازیکنان فراهم باشد.



Animations & audio (7)

هر کدام از بازیکنان باید براساس **Sprite** خود، انیمیشن‌های زیر را داشته باشند:



- Idle: حالت بی حرکت
- Move: حالت حرکت
- Attack: حالت حمله
- Death: حالت مرگ

همچنین هر بازیکن باید افکت‌های صوتی زیر را داشته باشد:



- صدای حمله
- صدای مرگ

بازی نیز باید دارای موسیقی پس‌زمینه (Background Music) و افکت‌های صوتی مختلف (Sound Effects) باشد.

8) following Enemy:

این دشمن در صورتی که بازیکنان وارد محدوده‌ی شناسایی‌اش شوند، آن‌ها را برای چند ثانیه دنبال می‌کند. اگر به بازیکن برسد، به او حمله کرده و آسیب (Damage) وارد می‌کند.



9) Shooting Enemy:

دشمن تیرانداز زمانی که بازیکنان وارد محدوده‌ی شناسایی‌اش شوند، روی یکی از آن‌ها Lock کرده و به سمت او شلیک می‌کند.



در صورتی که ایده‌ی خاصی برای دشمنان داشتید، می‌توانید آن را با TA ها مطرح کرده و پس از تأیید پیاده‌سازی کنید. برای مثال: دشمنی که پس از دیدن بازیکن، او را دنبال می‌کند و در صورت برخورد منفجر می‌شود. ایده‌های خلاقانه و خوب می‌توانند نمره‌ی امتیازی دریافت کنند.

10 Enemy health system & death system

هر دشمن باید یک **Health System** داشته باشد. با دریافت آسیب، مقدار سلامتی دشمن کم می‌شود و در صورت تمام شدن آن، دشمن می‌میرد.



11 Enemy Animation and audio

هر دشمن باید انیمیشن‌های زیر را داشته باشد:

- Idle: حالت بی حرکت
- Walking: حالت در حرکت
- Attacking: حالت حمله
- Death: حالت مرگ، زمانی که جان دشمن تمام شود.



12 Collectible ها:

Health Potion: هنگامی که یکی از بازیکنان این آیتم را برمی‌دارد، مقدار **Health Bar** او افزایش می‌یابد.
Buff Item: هنگامی که یکی از بازیکنان این آیتم را برمی‌دارد، یکی از ویژگی‌های او به صورت موقت افزایش پیدا می‌کند؛ برای مثال، میزان **Damage** وارد شده توسط بازیکن بیشتر می‌شود.



13 Puzzle ها:

Spike / Trap: تله‌هایی هستند که با قدم گذاشتن بازیکن یا حتی دشمنان روی آن‌ها، بسته به منطقی که شما تعریف می‌کنید، به **Health Bar** آن‌ها آسیب (**Damage**) وارد می‌کنند.



Triggers: دکمه‌هایی هستند که با رفتن روی آن‌ها فعال می‌شوند و می‌توانند یک در، مسیر یا مکانیزم خاص را باز کنند.



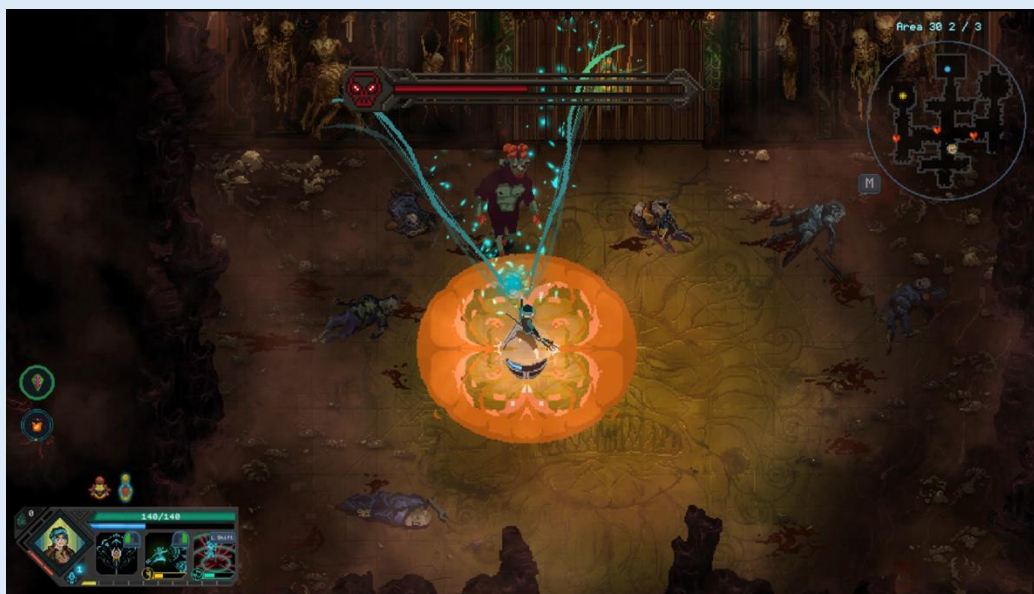
Keys: کلیدهایی هستند که با استفاده از آن‌ها می‌توان قفل‌ها یا درهای مشخصی را باز کرد.

14) Boss Fight:

باس باید یک دشمن چالش برانگیز باشد که در یک مرحله‌ی **Arena-like** با بازیکنان مبارزه می‌کند. باس باید به بازیکنان حمله کرده و به آن‌ها آسیب (**Damage**) وارد کند.

باس طراحی شده باید هنگام حمله روی یکی از بازیکنان **Lock** کند و به سمت او حمله کند. سپس رفتار خود را تغییر داده و بازیکن دیگر را هدف قرار دهد. همچنین باس باید **HealthBar** معقول داشته باشد تا به راحتی شکست نخورد.

دقت کنید که باس نباید شبیه دشمنان ساده‌ای باشد که در دو مرحله‌ی قبل طراحی کرده‌اید. باس باید پیچیدگی، الگوی حمله و رفتار متفاوتی نسبت به دشمنان معمولی داشته باشد.



15) Save System :

بازی باید دارای یک **Save System** باشد تا اطلاعات مهم بازیکنان ذخیره شود؛ از جمله:



- میزان **HP** بازیکنان
 - موقعیت بازیکنان
 - آخرین **Level**ی که بازیکنان در آن قرار داشته‌اند
- با این سیستم، بازیکنان مجبور نخواهند بود بازی را از ابتدا شروع کنند.

رابط کاربری مناسب برای بخش های مختلف بازی:

1. Main Menu :

صفحه ای که در ابتدای ورود به بازی با آن روبرو می شویم و باید دارای مقادیر زیر باشند:



A. Start game

B. Load

C. Settings

D. Quit

2. Pause Menu :

A. Resume

B. Settings

C. Save (در صورتیکه سیو به صورت خودکار

است (AutoSave)

(نیازی به این مورد نیست)

D. Main Menu

3. Settings Menu :

A. دکمه برای تغییر صدا و افکت های صوتی و صدای پس زمینه به صورت مجزا

B. دکمه Mute برای افکت های صوتی و صدای پس زمینه

4. Game over Screen :

A. نمایش متن Game Over

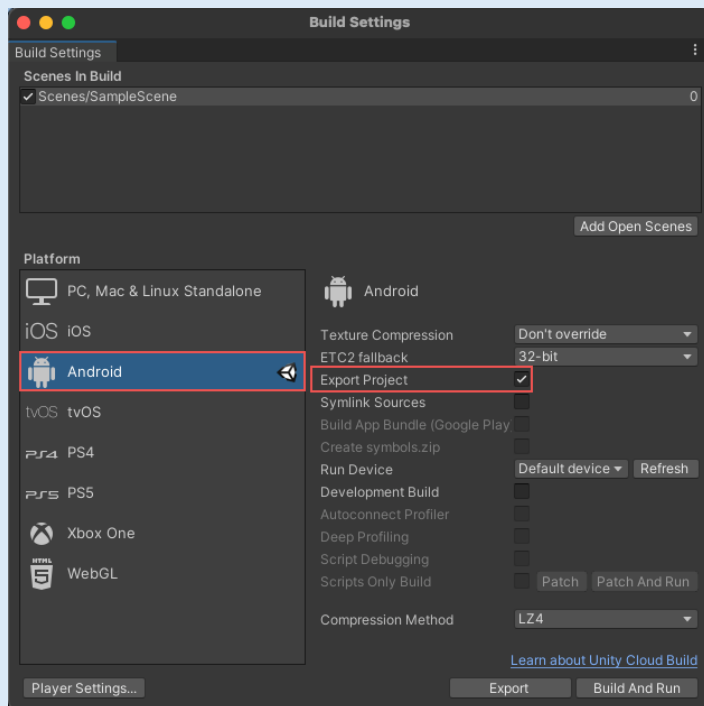
B. دکمه بازگشت به منو

C. دکمه شروع مجدد از آخرین سیو



17) Build گرفتن از بازی:

در نهایت، شما باید یک **Build** نهایی و قابل بازی از پروژه ارائه دهید. در فاز نهایی تحویل، بازی اجرا و بررسی می‌شود و نمره‌دهی براساس تجربه‌ی کامل بازی در نسخه‌ی **Build** انجام خواهد شد.



موارد امتیازی :

پیاده سازی اضافه این موارد نمره امتیازی خواهد داشت که می تواند n درصد به نمره کلی پروژه شما اضافه کند:

قابلیت تغییر اسکین بازیکن ها

قابلیت split screen و استفاده از Cinemachine

داشتن سیستم دیالوگ و استفاده مناسب از آن در بازی

داشتن داستان در بازی

توصیه ها:

- Scripting

- کد C# را تمیز، بهینه و همراه با توضیحات (کامنت‌های) مناسب بنویسید.
- از اصول برنامه نویسی شیگرا (OOP) به صورت موثر بهره بگیرید.
- در صورت نیاز، از الگوهای طراحی (Design Patterns) مناسب استفاده کنید.
- مدیریت صحنه ها (scene) را به صورت کارآمد انجام دهید
- برای کارکتر ها دشمنان آتیم های قابل جمع آوری (Collectibles) و سایر اجزا از Prefab به طور گسترده استفاده کنید.

- Art & assets: سعی کنید از Asset هایی که در اختیارتان قرار داده شده بهترین استفاده را ببرید.

- Performance: بهینه سازی عملکرد را در نظر داشته باشید در موقعیت های مناسب، از تکنیک های بهینه سازی

پایه مانند Object Pooling برای کاهش هزینه های پردازشی استفاده کنید.

- عملکرد بازی شما باید مستقل از فریم‌ریت بازی باشد و سرعت بازی باید کم یا زیاد شود.

- Physics: یک سیستم فیزیک دو بعدی قابل اعتماد با استفاده از RigidBody2D و Collider ها پیاده سازی

کنید. در صورت نیاز، از تنظیمات Physics Materials برای کنترل اصطکاک استفاده کنید.

- Boss Fight: سعی کنید برای طراحی اکشن های Boss از استیت ماشین ساده استفاده کنید

- Animation: انیمیشن های sprite را برای حالت های مختلف کاراکتر در هر دو نقش بازیکن و دشمن پیاده سازی

کنید.

- از Animator Controller و ترنزشن ها (Transitions) به صورت موثر برای مدیریت حالات انیمیشن

استفاده شود.

دقت کنید اهمیت نرم (Smooth) بودن تجربه بازی از ویژگی (feature) ها و مکانیک های متعدد بسیار مهم تر است چه بسا

که تیمی بازی با گیمپلی بهتر نمره بیشتری از تیمی با فیچر ها و خط های کد بیشتر بگیرد.

به دلیل محدودیت در Asset ها و Art ها زیبایی بازی اهمیت کمتری دارد با اینحال انتظار می رود از asset هایی که در

اختیارتان قرار داده شده به خوبی استفاده کنید.